

Lab 2 — Task 7: Continuous Deployment with Kubernetes

Akash Singh Sagar | Repository: github.com/akashsinghsagar/devops-lab2

Objective

Integrate the Kubernetes deployment with a CI/CD pipeline so that a push to the main branch automatically builds the application image, applies the Kubernetes manifests and verifies that the rollout succeeded. GitHub Actions is used as the pipeline.

Pipeline overview

Stage	What happens	Tool
Trigger	Push to main that changes app/, k8s/ or the workflow file (or a manual run)	GitHub Actions
Cluster	A Minikube cluster is started inside the runner	medyagh/setup-minikube
Build	The image is built directly into the cluster, tagged with the short commit SHA	minikube image build
Configure	The image tag in k8s/deployment.yaml is replaced with the new SHA tag	sed
Deploy	All manifests in k8s/ are applied	kubectl apply -f k8s/
Verify	Rollout status is awaited and /health is called through the Service	kubectl, curl

Step 1 — Repository layout

The manifests from Task 5 live in k8s/, and the pipeline definition is in .github/workflows/cd.yml. Everything the deployment needs is version-controlled, so a commit is the only action required to deploy.

```
devops-lab2/
+-- app/                # app.py, requirements.txt, Dockerfile
+-- k8s/
|   +-- deployment.yaml
|   +-- service.yaml
+-- .github/
|   +-- workflows/
|       +-- cd.yml
```

Step 2 — Write the CD workflow

The workflow runs on every relevant push to main. Using the commit SHA as the image tag means every commit produces a different pod template, so each run triggers a genuine rolling update. kubectl apply is declarative and idempotent: unchanged objects are left alone, changed ones are updated.

.github/workflows/cd.yml

```
name: CD - Deploy to Kubernetes
```

```

on:
  push:
    branches: [main]
    paths: ["app/**", "k8s/**", ".github/workflows/cd.yml"]
  workflow_dispatch:

jobs:
  deploy:
    runs-on: ubuntu-latest
    steps:
      - name: Checkout
        uses: actions/checkout@v4

      - name: Start Minikube
        uses: medyagh/setup-minikube@latest
        with:
          driver: docker

      - name: Build image inside Minikube
        run: |
          echo "TAG=${GITHUB_SHA:7}" >> $GITHUB_ENV
          minikube image build -t lab2-python-app:${GITHUB_SHA:7} ./app

      - name: Point manifest at the new image
        run: |
          sed -i "s|\\(image: .*:\\).*|\\1${TAG}|" k8s/deployment.yaml
          grep image: k8s/deployment.yaml

      - name: Apply manifests
        run: kubectl apply -f k8s/

      - name: Wait for rollout
        run: |
          kubectl rollout status deployment/lab2-app --timeout=180s
          kubectl get deployments,pods,svc

      - name: Smoke test
        run: |
          URL=$(minikube service lab2-app-svc --url)
          curl -sf $URL/health

```

Note on the cluster. The Minikube cluster in this workflow lives inside the GitHub-hosted runner and is discarded when the job finishes, so the run demonstrates the whole automated flow (build → apply → verify). To deploy to the persistent Minikube cluster on the local machine instead, register that machine as a self-hosted runner, change runs-on to self-hosted and remove the “Start Minikube” step; the remaining steps stay the same.

Step 3 — Commit and push to trigger the pipeline

```
git add .github/workflows/cd.yml k8s/  
git commit -m "Add Kubernetes CD workflow"  
git push origin main
```

The screenshot shows a GitHub repository interface. At the top, a commit message "Arsh Add Task 5-7 app, Kubernetes manifests, and CD workflow" is displayed with a commit hash "12a4b36" and a "now" timestamp. Below this, a file list shows the repository structure: `.github/workflows`, `app`, and `k8s` folders, and several PDF files and a `README.txt` and `run-k8s-tasks.ps1` file. The `README` file is expanded, showing setup instructions and a task list. The right sidebar shows repository statistics: 0 stars, 0 watching, 0 forks, and a list of languages: PowerShell (85.1%), Python (11.9%), and Dockerfile (3%).

Commit: Arsh Add Task 5-7 app, Kubernetes manifests, and CD workflow (12a4b36 · now)

Files:

- `.github/workflows`: Add Task 5-7 app, Kubernetes manif... (now)
- `app`: Add Task 5-7 app, Kubernetes manif... (now)
- `k8s`: Add Task 5-7 app, Kubernetes manif... (now)
- `Lab-2-Task-1.pdf`: Add files via upload (2 weeks ago)
- `Lab-2-Task-2.pdf`: Add files via upload (2 weeks ago)
- `Lab-2-Task-3.pdf`: Add files via upload (2 weeks ago)
- `Lab-2-Task-4.pdf`: Add files via upload (2 weeks ago)
- `README.txt`: Add Task 5-7 app, Kubernetes manif... (now)
- `run-k8s-tasks.ps1`: Add Task 5-7 app, Kubernetes manif... (now)

README

SETUP

1. Copy `k8s/` and `.github/` into your repo (next to `app/`).
Note: `k8s/deployment.yaml` and the workflow assume the image is `lab2-python-app` and the Dockerfile is in `app/`. Adjust if yours differs.
2. Docker Desktop and Minikube must be running.
3. From the repo root: `powershell -ExecutionPolicy Bypass -File .\run-k8s-tasks.ps1`
4. At each yellow prompt press Win+Shift+S, capture the terminal, paste into the matching box in the Word file, then press Enter.

TASK 7 (CD) - after the scripts:

```
git add .github k8s  
git commit -m "Add Kubernetes CD workflow"  
git push origin main
```

4. Screenshot the Actions tab (Figures 1-3), then edit `app/app.py`, push again for Figure 4.

For Task-8 Figure 7 reuse a successful Actions run screenshot.

Releases
No releases published
[Create a new release](#)

Packages
No packages published
[Publish your first package](#)

Contributors 1
akashsinghsagar AKASH SI...

Languages

- PowerShell 85.1%
- Python 11.9%
- Dockerfile 3%

Figure 1 — The push to main starts the CD workflow automatically; the run completes successfully.

Step 4 — Inspect the pipeline run

Opening the run shows each step of the job. All steps — checkout, Minikube start, image build, manifest update, apply, rollout wait and smoke test — complete with a green check.

```
C:\Users\ARSH\OneDrive\Desktop\PDFs\zeee\labssss\lab2-k8s-kit>git add .
warning: in the working copy of '.github/workflows/cd.yml', LF will be replaced by CRLF the next time Git touches it
warning: in the working copy of 'README.txt', LF will be replaced by CRLF the next time Git touches it
warning: in the working copy of 'app/app.py', LF will be replaced by CRLF the next time Git touches it
warning: in the working copy of 'k8s/deployment.yaml', LF will be replaced by CRLF the next time Git touches it
warning: in the working copy of 'k8s/service.yaml', LF will be replaced by CRLF the next time Git touches it
warning: in the working copy of 'run-k8s-tasks.ps1', LF will be replaced by CRLF the next time Git touches it

C:\Users\ARSH\OneDrive\Desktop\PDFs\zeee\labssss\lab2-k8s-kit>git commit -m "Add Task 5-7 app, Kubernetes manifests, and CD workflow"
[main 12a4b36] Add Task 5-7 app, Kubernetes manifests, and CD workflow
 8 files changed, 255 insertions(+)
 create mode 100644 .github/workflows/cd.yml
 create mode 100644 README.txt
 create mode 100644 app/Dockerfile
 create mode 100644 app/app.py
 create mode 100644 app/requirements.txt
 create mode 100644 k8s/deployment.yaml
 create mode 100644 k8s/service.yaml
 create mode 100644 run-k8s-tasks.ps1

C:\Users\ARSH\OneDrive\Desktop\PDFs\zeee\labssss\lab2-k8s-kit>git push origin main
Enumerating objects: 15, done.
Counting objects: 100% (15/15), done.
Delta compression using up to 12 threads
Compressing objects: 100% (11/11), done.
Writing objects: 100% (14/14), 4.29 KiB | 732.00 KiB/s, done.
Total 14 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)
To https://github.com/akashsinghsagar/devops-lab2.git
 3f94ded..12a4b36  main -> main
```

Figure 2 — Every pipeline step succeeds, from checkout to the smoke test.

Step 5 — Verify that the manifests were applied

The log of the “Apply manifests” and “Wait for rollout” steps shows the effect of `kubectl apply` and the resulting cluster state: the Deployment and Service are created, the rollout completes, and the pods are Running.

Step 6 — Trigger an automatic redeploy

To show that a code change is deployed without any manual `kubectl` command, the greeting message in the application is changed and pushed.

```
git add app/app.py
git commit -m "Update greeting message"
git push origin main
```

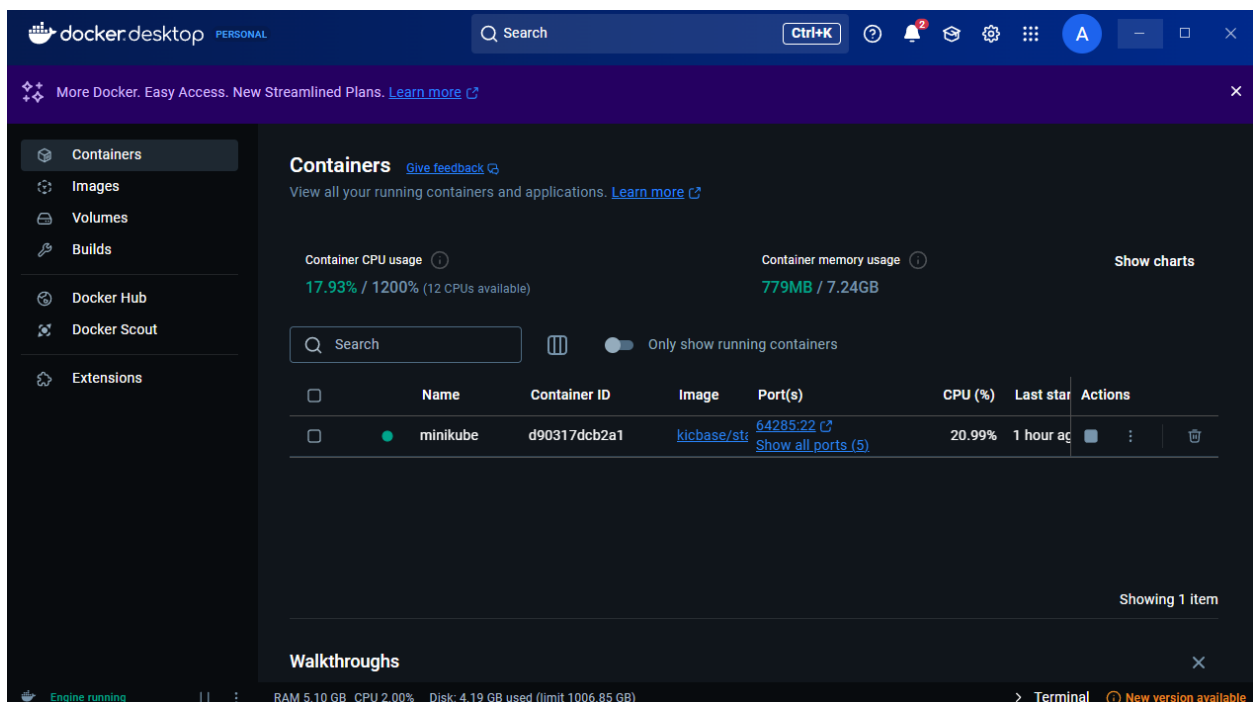
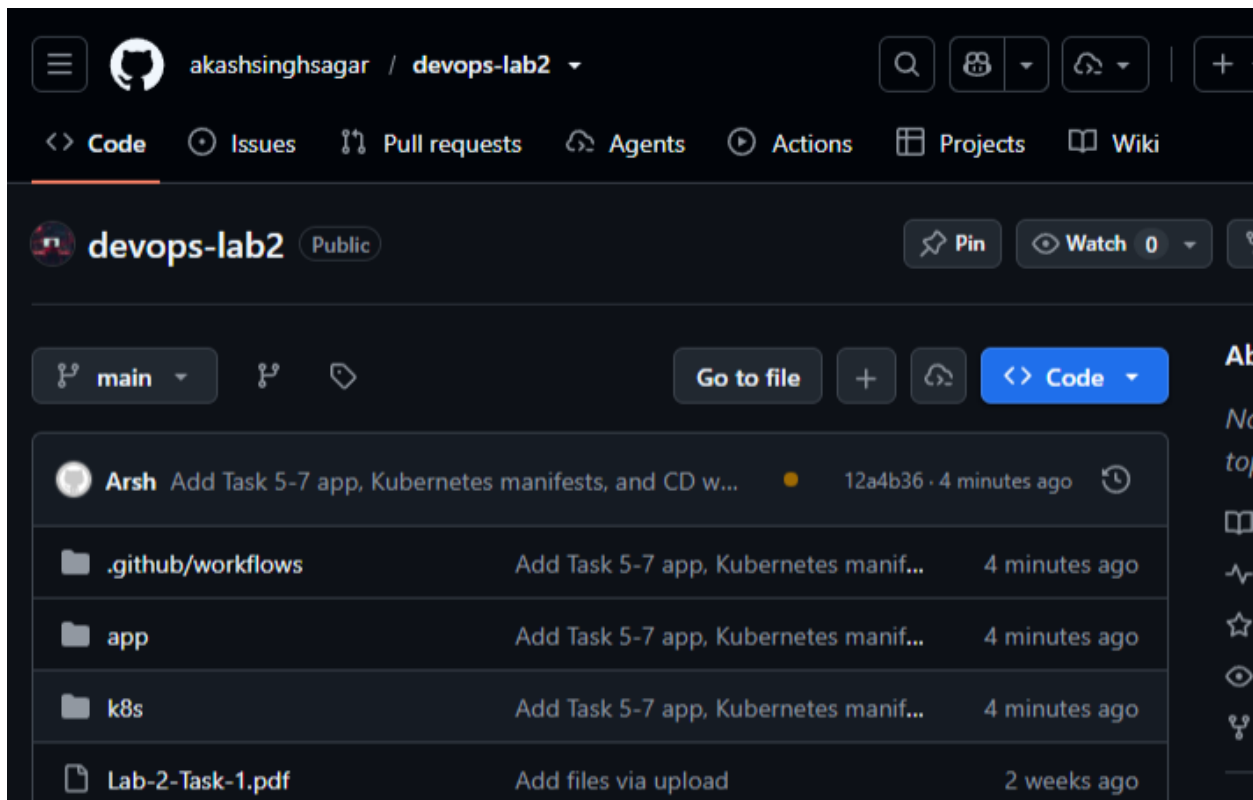


Figure 4 — A second run starts by itself; the manifest now references a new SHA-based image tag, and the Deployment is rolled out with the new image.

Result

Deployments to Kubernetes are now automated: a push to main starts a pipeline that builds a uniquely tagged image, applies the manifests with `kubect`l apply, waits for the rollout to complete and smoke-tests the `/health` endpoint. A second push produced a second automatic rollout with a new image tag, without any manual `kubect`l command. Task 7 is complete.